Universidade de São Paulo
Instituto de Matemática,
Estatística e Ciência da Computação
Bacharelado em Ciência da Computação

Desenvolvimento de ferramenta de ensino adaptativo de shogi com feedback para iniciantes

Vinícius Yuiti Okuma Shimizu

Monografia Final

mac 499 — Trabalho de Formatura Supervisionado

Supervisor: Dr. Denis Deratani Mauá

São Paulo
2026

O conteúdo deste trabalho é publicado sob a licença CC BY 4.0
(Creative Commons Attribution 4.0 International License)

Agradecimentos

Do. Or do not. There is no try.
— Mestre Yoda

Texto texto texto

Resumo

Vinícius Yuiti Okuma Shimizu. **Desenvolvimento de ferramenta de ensino adaptativo de shogi com feedback para iniciantes**. Monografia (Bacharelado). Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, em forma de texto. Deve apresentar os objetivos, métodos empregados, resultados e conclusões. O resumo deve ser redigido em parágrafo único, conter no máximo 500 palavras e ser seguido dos termos representativos do conteúdo do trabalho (palavras-chave). Deve ser precedido da referência do documento. Texto texto texto

Abstract

Vinícius Yuiti Okuma Shimizu. **Development of Shogi Teaching Tool With Adaptative Feedback For Beginners**. Capstone Project Report (Bachelor). Institute of Mathematics, Statistics, and Computer Science, University of São Paulo, São Paulo, 2026.

Elemento obrigatório, elaborado com as mesmas características do resumo em língua portuguesa. De acordo com o Regimento da Pós-Graduação da USP (Artigo 99), deve ser redigido em inglês para fins de divulgação. É uma boa ideia usar o sítio www.grammarly.com na preparação de textos em inglês. Text text text

Sumário

1 Introdução **1**

2 Conceitos Fundamentais **3**

2.1 Shogi . . . 3

2.2 Convenções de shogi . . . 3

2.3 Representação SFEN . . . 5

2.4 Universal Shogi Interface . . . 7

3 Trabalhos relacionados **9**

3.1 Sistema de apoio ao aprendizado de shogi . . . 9

3.2 Sistema tutor inteligente para jogos estratégicos . . . 9

3.3 Sistema de ensino de xadrez com grafo de conhecimento . . . 10

3.4 Korbit . . . 11

3.5 Comparação com o Shogi-sensei . . . 12

4 Desenvolvimento **13**

4.1 Banco de dados . . . 14

4.2 *Engine* . . . 15

4.3 Servidor . . . 17

4.3.1 Arquitetura . . . 17

4.3.2 Autenticação . . . 18

4.3.3 Geração automática de exercícios . . . 18

4.3.4 Listas de exercício . . . 20

4.3.5 Interface . . . 22

5 Metodologia **25**

6 Resultados **27**

7 Conclusão **29**

Referências **31**

Capítulo 1

Introdução

Shogi é um jogo de estratégia japonês semelhante ao xadrez, mas que apresenta mecânicas adicionais que aumentam a dificuldade de seu aprendizado. Entre essas mecânicas, destacam-se a possibilidade de reutilizar peças capturadas e a promoção de peças para versões com movimentos diferentes.

Ao longo dos anos, plataformas como o Lishogi surgiram com o intuito de facilitar o acesso ao aprendizado, oferecendo recursos como partidas online ou resolução de problemas como formas de treino. Essas plataformas, no entanto, apresentam limitações que se tornam uma barreira ao aprendizado de iniciantes. O Lishogi, por exemplo, disponibiliza exercícios simples para quem está aprendendo, mas não fornece informações sobre os motivos pelos quais uma resposta está incorreta. O mesmo vale para as análises de partidas, nas quais as jogadas recebem uma pontuação de acordo com sua qualidade, sem explicar por que determinada decisão seria melhor.

Para um iniciante que deseja aprender a jogar sozinho, a falta de orientação pode dificultar a compreensão dos conceitos envolvidos e assim limitar o aproveitamento dos exercícios realizados. Nesse contexto, o uso de modelos de linguagem pode contribuir para suprir tal necessidade ao fornecer ao jogador explicações moldadas ao seu nível de conhecimento, desempenhando um papel semelhante ao de um tutor.

Diante desse cenário, este trabalho propõe o desenvolvimento do Shogi-sensei, uma plataforma web voltada ao ensino introdutório de shogi. O projeto fornece exercícios divididos em módulos que abordam os conceitos fundamentais do jogo: reconhecimento das peças, movimentação, promoção, reintrodução de peças capturadas e situações de xeque-mate. Ao final de cada lista, o usuário recebe os resultados, incluindo as questões respondidas incorretamente e uma análise gerada por um modelo de linguagem (LLM) que explica os motivos do erro. A seleção de exercícios de cada lista é adaptada ao desempenho do jogador, aumentando a proporção de exercícios dos módulos nos quais ele apresenta maior dificuldade. Para avançar para o próximo módulo, é necessário atingir uma média mínima nas listas mais recentes do módulo atual.

A avaliação do sistema foi realizada durante 2 semanas com X usuários. Durante esse período, foram registradas métricas como taxa de acerto, tempo de resposta e distribui-

ção de exercícios por módulo para acompanhar o desempenho dos usuários ao longo do experimento.

O restante deste trabalho está organizado da seguinte forma. O capítulo 2 apresenta os conceitos e tecnologias utilizados para desenvolver este trabalho. O capítulo 3 apresenta trabalhos relacionados. O capítulo 4 descreve a implementação do sistema. O capítulo 5 contém a metodologia utilizada para avaliar a plataforma desenvolvida. O capítulo 6 apresenta e discute os resultados obtidos no experimento descrito no capítulo anterior. Por fim, o capítulo 7 apresenta as conclusões e possíveis implementações futuras.

Capítulo 2

Conceitos Fundamentais

2.1 Shogi

O shogi é um jogo de tabuleiro disputado em um tabuleiro com 9 linhas e 9 colunas. O jogo utiliza 8 tipos de peças, das quais 6 podem assumir formas promovidas (que possuem movimentações diferentes das versões padrão), totalizando 14 tipos de peças. A notação ocidental das peças e seus respectivos símbolos japoneses são apresentados na Tabela 2.1.

A notação ocidental utilizada em publicações de língua inglesa foi desenvolvida por George Hodges (1976) [Hodges, s.d.], com base em elementos da notação algébrica do xadrez.

2.2 Convenções de shogi

Símbolo	Notação ocidental	Peça
歩	P	Peão
香	L	Lança
桂	N	Cavalo
銀	S	General de Prata
金	G	General de Ouro
角	B	Bispo
飛	R	Torre
玉	K	Rei
と	+P	Peão Promovido
成香	+L	Lança Promovida
成桂	+N	Cavalo Promovido
成銀	+S	General de Prata Promovido
馬	+B	Bispo Promovido
龍 ou 竜	+R	Torre Promovida

Tabela 2.1: *Correspondência entre as notações japonesa e ocidental das peças de shogi*

- **Jogadores**: os jogadores são definidos como preto (*sente*), quem faz o primeiro movimento, ou branco (*gote*).
- **Casas**: as colunas são numeradas de 1 a 9, da direita para a esquerda, e as linhas são identificadas com letras de a a i, de cima para baixo, considerando o ponto de vista do sente.
- **Posição**: uma casa é identificada pela combinação do número da coluna com a letra da linha. Por exemplo, 5a corresponde à casa localizada na coluna 5 e na primeira linha.

Peça	Movimento
Peão	Uma casa para a frente.
Lança	Qualquer número de casas para a frente.
Cavalo	Salta para uma das duas casas duas fileiras à frente e uma coluna para um dos lados.
General de prata	Uma casa na diagonal ou uma casa para a frente.
General de ouro	Uma casa para a frente, para trás ou para os lados, ou uma casa na diagonal para a frente.
Bispo	Qualquer número de casas na diagonal.
Torre	Qualquer número de casas na horizontal ou vertical.
Rei	Uma casa em qualquer direção.
Peão promovido	Como um general de ouro.
Lança promovida	Como um general de ouro.
Cavalo promovido	Como um general de ouro.
General de prata promovido	Como um general de ouro.
Bispo promovido	Como um bispo ou uma casa na horizontal ou vertical.
Torre promovida	Como uma torre ou uma casa na diagonal.

Tabela 2.2: *Movimentos das peças*

- **Movimentos**[Gilliard, s.d.]: o sistema adota uma variação da convenção de Hodges, com adaptações para facilitar a compreensão por iniciantes. Os movimentos de cada peça se encontram na Tabela 2.2. Um movimento é considerado legal quando segue as regras do jogo descritas abaixo.
  - **Deslocamento**: [Origem]->[Destino]. Por exemplo, **9g->9f** se refere ao movimento da casa **9g** para a casa **9f**. Na notação de referência, o deslocamento não possui o símbolo ->".
  - **Captura**: quando aplicável, o movimento ocupa a casa da peça adversária capturada, que é removida do tabuleiro e passa a pertencer à mão jogador que realizou a captura.
  - ***Drop***: [Peça]*[Posição]. Refere-se à recolocação no tabuleiro de uma peça capturada anteriormente. Por exemplo, **P*9e** representa a colocação de um peão na casa **9e**.

  - **Promoção**: [Origem]->[Destino]+. O símbolo ”+”representa que a peça foi promovida ao final do movimento. Por exemplo, **9d->9c+** representa o movimento da casa **9d** para a casa **9c**, seguido de sua promoção. A promoção pode ocorrer quando a origem ou destino do movimento está dentro da zona de promoção, que corresponde às três fileiras mais distantes do jogador. Em determinadas situações ela é obrigatória: por exemplo, um peão ou uma lança que alcança a última fileira, ou um cavalo que alcança uma das duas últimas fileiras, pois não possuem movimentos legais a partir dessas posições.

- **Xeque**: Um Rei está em **xeque** quando pode ser capturado por uma ou mais peças do oponente. O jogador ameaçado deve realizar um movimento legal que elimine essa ameaça.

- **Xeque-mate**: Um Rei está em **xeque-mate** quando ele está em xeque e não existe um movimento legal que elimine o xeque. Nessa situação, a vitória pertence ao jogador que causou o xeque-mate.

2.3 Representação SFEN

No shogi, a *Shogi Forsyth–Edwards Notation* (SFEN)[Romstadt, s.d.] é utilizada para representar textualmente uma posição no jogo. Essa representação é composta por quatro campos separados por espaços:

- Tabuleiro: disposição das peças nas casas;

- Turno: jogador que deve realizar o próximo movimento, b para o *sente* e w para o *gote*;

- Peças nas mãos: peças capturadas que podem ser reutilizadas por quem fez a captura;

- Número do movimento: contador do movimento atual.

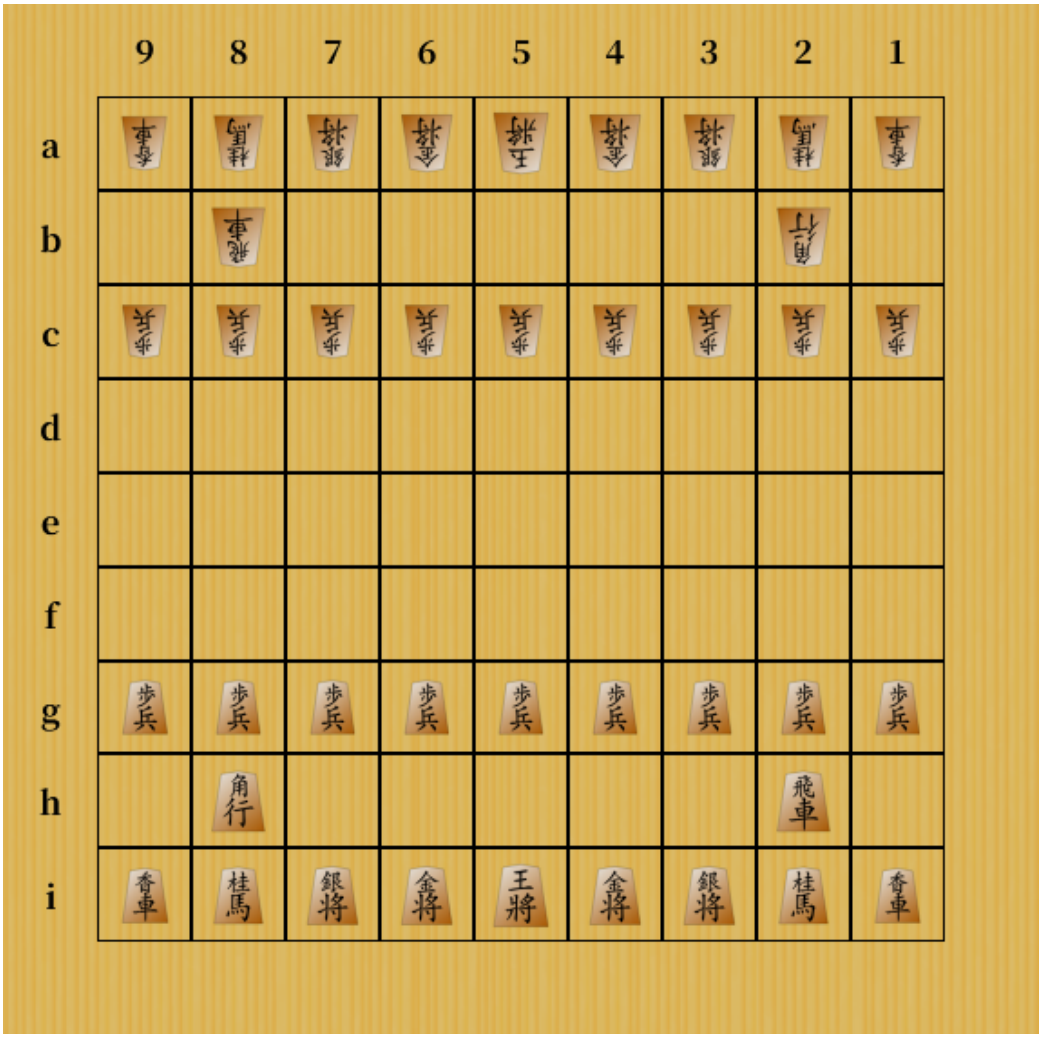


Figura 2.1: *Posição inicial do tabuleiro*

Considere a seguinte representação SFEN correspondente à posição inicial do tabuleiro:

```
lnsgkgsnl/1r5b1/ppppppppp/9/9/9/PPPPPPPPP/1B5R1/LNSGKGSNL b - 1
```

Como ilustrado na Figura 2.1, o primeiro campo descreve o tabuleiro, cujas linhas são separadas pelo caractere ”/”e estão ordenadas de cima para baixo. As peças seguem a notação da Tabela 2.1. As letras minúsculas representam as peças do *gote* e as maiúsculas representam as peças do *sente*. Casas vazias são representadas por números que indicam a quantidade de casas consecutivas desocupadas.

O segundo campo indica que o *sente* deve fazer o próximo movimento, uma vez que ele é o jogador que começa o jogo. O símbolo - no terceiro campo indica que nenhum deles possui peças capturadas disponíveis para reutilização. Por fim, o quarto campo indica que a próxima jogada será a primeira do jogo.

2.4 Universal Shogi Interface

No contexto do shogi computacional, os chamados motores, ou *engines*, são programas capazes de analisar posições e buscar movimentos possíveis, utilizando algoritmos de busca e funções de avaliação para estimar a qualidade de diferentes jogadas. Esses motores podem ser integrados a interfaces gráficas, permitindo o desenvolvimento de aplicações de análise de partidas, como o Lishogi, que utiliza o YaneuraOu[*Repositório do YaneuraOu* s.d.].

A comunicação entre motores e interfaces pode ser realizada por meio do protocolo *Universal Shogi Interface* (USI), baseado no protocolo equivalente utilizado no xadrez *Universal Chess Interface* (UCI). Ao padronizar comandos e respostas, o USI elimina a necessidade de implementar os mecanismos de análise dentro das próprias aplicações.

Esse protocolo define operações para inicializar o motor, configurar suas opções, fornecer posições do tabuleiro pro meio da representação *SFEN* e solicitar a busca por movimentos. Essa comunicação ocorre através da entrada e da saída padrão do processo do motor. Durante uma busca, o programa pode retornar informações sobre a análise realizada, como a profundidade alcançada que indica quantos meios-lances (movimentos individuais de um jogador) foram explorados na busca, a avaliação da posição e os movimentos candidatos considerados.

Capítulo 3

Trabalhos relacionados

3.1 Sistema de apoio ao aprendizado de shogi

Ito (2018)[Ito, 2018] propôs um método de apoio ao aprendizado de shogi para jogadores iniciantes, denominado *Posição Futura*. O autor observou que a apresentação explícita de movimentos recomendados por programas de shogi pode levar os jogadores a dependerem excessivamente das sugestões do computador, prejudicando o desenvolvimento de uma habilidade importante em jogos de estratégia: antecipar possíveis jogadas para tomar decisões. Para investigar como diferentes níveis de suporte afetam o aprendizado, ele desenvolveu um sistema que apresenta posições futuras do tabuleiro, geradas por meio de partidas entre programas de inteligência artificial (IA), considerando diferentes distâncias (jogadas à frente) em relação à posição atual. Dessa forma, o jogador recebe informações que podem auxiliar na compreensão do desenvolvimento da partida sem que uma jogada específica seja indicada diretamente.

O autor avaliou a proposta por meio de um experimento que comparou diferentes níveis de assistência durante a prática de shogi, incluindo uma condição sem assistência. Os resultados indicaram uma tendência de melhora nos testes após a utilização das dicas, embora o número reduzido de participantes não tenha permitido identificar diferenças estatisticamente significativas entre os níveis de assistência. O estudo sugere, portanto, que a apresentação de informações sobre o jogo pode constituir uma abordagem promissora para apoiar o aprendizado de shogi, embora sejam necessárias investigações adicionais.

3.2 Sistema tutor inteligente para jogos estratégicos

No trabalho de Zhang *et al.* (2025)[Q. Zhang *et al.*, 2025], os autores observaram que a abordagem padrão utilizada por tutores humanos de jogos estratégicos, como Go, Gomoku e Shogi, foi expor jogadores iniciantes a diversas partidas e ao final analisar as jogadas realizadas. Dessa forma, investigaram a possibilidade de desenvolver um sistema tutor inteligente que permite que os jogadores aprendam a partir da revisão dos jogos, comparando essa estratégia com outros dois métodos de suporte: a apresentação de dicas ao longo da partida e o fornecimento de feedback imediato após cada jogada.

Nesse contexto, foi proposto um sistema que permite ao usuário jogar Gomoku contra uma IA treinada no jogo e fornece a ele as abordagens de ensino mencionadas anteriormente. Para avaliar o sistema, foram conduzidos dois estudos-piloto nos quais os participantes foram distribuídos aleatoriamente entre as três condições de aprendizagem. No primeiro estudo, após interagirem com o sistema durante 20 minutos, os participantes jogaram cinco partidas de teste. Os resultados indicaram que aqueles que receberam feedback imediato apresentaram melhor aprendizagem do que aqueles que receberam feedback tardio. No segundo estudo, foi adotado um formato de pré-teste e pós-teste: os participantes responderam inicialmente a 20 questões sobre estratégias de Gomoku, utilizaram o sistema durante 35 minutos e, em seguida, responderam às mesmas questões em ordem diferente, com os tabuleiros rotacionados ou espelhados. Os resultados preliminares indicaram que os participantes de todas as condições obtiveram desempenho significativamente superior no pós-teste em relação ao pré-teste.

3.3 Sistema de ensino de xadrez com grafo de conhecimento

No trabalho de Zhang *et al.* (2024)[M. Zhang *et al.*, 2024], os autores identificaram três desafios enfrentados pelas ferramentas de ensino de xadrez: a atenção limitada dedicada ao ensino de finais de partida; a escassez de sistemas de ensino de xadrez baseados em grafos de conhecimento, estruturas que representam fatos por meio de entidades, relações e descrições semânticas; e a ausência de critérios padronizados para avaliar os resultados de aprendizagem dos participantes em processos de ensino de xadrez que utilizam inteligência artificial.

Para abordar essas questões, os autores desenvolveram o sistema chamado *EK-Chess*, que permite aos usuários jogar xadrez e receber feedback em tempo real. O sistema utiliza um grafo de conhecimento de finais de partida, construído a partir do conhecimento de especialistas, no qual as posições são representadas por meio da notação Forsyth-Edwards (FEN) e os movimentos pela notação algébrica longa *Long Algebraic Notation* (LAN). Esses elementos são organizados como nós de um grafo direcionado, que serve de base para fornecer informações sobre movimentos e explicações relacionadas a cenários específicos do jogo.

Para avaliar o sistema, os autores conduziram um experimento com 20 crianças entre 6 e 10 anos de idade, que conheciam os movimentos das peças e as regras do xadrez, mas ainda não dominavam estratégias mais avançadas, sendo consideradas no nível iniciante. Os participantes foram divididos em dois grupos, um que estudou usando o *EK-Chess* e outro usando livros tradicionais. Após aprenderem a utilizar os respectivos recursos de ensino, os participantes de ambos os grupos responderam a um teste antes e outro depois do período de estudo. As questões apresentavam diferentes níveis de dificuldade, definidos de acordo com o conhecimento necessário para chegar à resposta correta.

Os resultados indicaram que os participantes que utilizaram o EK-Chess obtiveram melhor desempenho no pós-teste, especialmente nas questões de dificuldade intermediária. No grupo que utilizou livros, também foi observado um desempenho melhor nessas ques-

tões, mas sem uma melhora estatisticamente significativa no resultado geral, conforme relatado pelos autores. Os resultados, portanto, sugerem que o EK-Chess pode contribuir para o aprendizado de estratégias de xadrez.

Os autores argumentam que uma das principais diferenças entre os métodos de ensino está na presença de um mecanismo de registro do progresso no sistema, que permite aos usuários identificar as etapas já concluídas e aquelas que ainda precisam ser realizadas. Segundo os autores, esse mecanismo pode reduzir a carga cognitiva associada ao acompanhamento do próprio aprendizado, contribuindo para a eficácia do processo de ensino. Além disso, discutem a possibilidade de utilizar os dados registrados durante o aprendizado em conjunto com sistemas de recomendação, permitindo modelar as preferências e o domínio do conteúdo por cada usuário para oferecer recomendações de estudo personalizadas e, potencialmente, melhorar os resultados de aprendizagem.

Por fim, os autores exploram a possibilidade de utilizar o grafo de conhecimento desenvolvido para fornecer informações a modelos de linguagem de grande escala (*Large Language Models* —LLMs), visando aprimorar seu desempenho em tarefas relacionadas ao xadrez.

3.4 Korbit

Kochmar *et al.* (2020)[Kochmar *et al.*, 2020] investigaram o uso de técnicas automatizadas para selecionar feedback em sistemas tutores inteligentes, considerando as características das orientações disponíveis e os dados relacionados ao desempenho e às interações anteriores dos estudantes. Os autores buscaram avaliar se a personalização desse processo poderia contribuir para melhorar os resultados de aprendizagem.

Para avaliar essa abordagem, os autores realizaram um experimento com 183 estudantes de *Machine Learning* (ML) que precisaram utilizar uma plataforma de ensino online chamada *Korbit*[*Plataforma Korbit* s.d.] (um sistema tutor inteligente baseado em diálogo que emprega técnicas ML, processamento de linguagem natural (PLN) e aprendizado por reforço para oferecer uma experiência de aprendizagem interativa e personalizada) durante um mês.

Os autores investigaram três tipos de feedback automático: dicas e explicações geradas por meio de técnicas de processamento de linguagem natural (PLN); explicações extraídas de artigos da Wikipédia sobre aprendizado de máquina; e dicas para problemas matemáticos, incluindo equações incompletas e orientações para corrigir equações incorretas. Para selecionar as dicas personalizadas, foram avaliados três modelos: *baseline*, que considera características linguísticas das dicas, como seu tamanho e a sobreposição de palavras-chave com a pergunta; *shallow personalization*, que utiliza características relacionadas ao desempenho do estudante, como a quantidade de questões tentadas e a proporção de respostas corretas e incorretas; e *deep personalization*, que também considera características linguísticas de até quatro interações anteriores entre o estudante e o sistema.

O ganho de aprendizagem foi avaliado pela proporção de questões inicialmente respondidas incorretamente que passaram a ser respondidas corretamente após o recebimento de uma dica ou explicação. Os autores analisaram essa métrica considerando to-

dos os estudantes e, separadamente, aqueles que receberam feedback antes da segunda tentativa. Os resultados indicaram que o modelo *deep personalization* apresentou o maior ganho de aprendizagem, enquanto o *baseline* apresentou o menor, na análise que considerou todos os estudantes. Na análise restrita aos estudantes que receberam feedback antes da segunda tentativa, a diferença entre o modelo *deep personalization* e o *baseline* foi estatisticamente significativa. Esses resultados sugerem que considerar informações sobre o desempenho e as interações anteriores dos estudantes pode contribuir para a eficácia do feedback em sistemas tutores inteligentes.

3.5 Comparação com o Shogi-sensei

Embora os trabalhos apresentados proponham abordagens distintas, eles compartilham o objetivo de utilizar recursos computacionais para apoiar o processo de aprendizagem. Nesse contexto, o Shogi-sensei também busca auxiliar jogadores iniciantes, mas concentra-se no ensino de fundamentos específicos do shogi por meio de listas de exercícios. Diferentemente do sistema de Ito, que auxilia o jogador a compreender possíveis desenvolvimentos de uma partida, o Shogi-sensei organiza o aprendizado em exercícios direcionados a habilidades específicas. Em relação ao tutor de gomoku, que permite aprender por meio de partidas contra uma inteligência artificial, o Shogi-sensei prioriza a resolução de exercícios individuais, sem exigir que o usuário participe de uma partida completa.

Também existem diferenças nos mecanismos de suporte ao aprendizado. O EK-Chess utiliza um grafo de conhecimento construído a partir do conhecimento de especialistas para fornecer informações sobre movimentos e explicações relacionadas a posições específicas do xadrez. Já o Korbit investiga métodos de seleção automatizada de feedback que consideram características linguísticas das orientações, informações sobre o desempenho dos estudantes e dados de interações anteriores. O Shogi-sensei, por sua vez, apresenta explicações apenas no final das listas de exercícios e registra informações como acertos e tempo de resposta, utilizando o desempenho dos usuários para orientar a recomendação de exercícios de determinados módulos de aprendizagem.

Por fim, os trabalhos também diferem quanto às estratégias de avaliação. Os estudos apresentados empregaram experimentos com diferentes condições de aprendizagem, testes anteriores e posteriores às atividades ou métricas relacionadas à resolução de exercícios após o recebimento de feedback. Neste trabalho, a avaliação considera métricas como a taxa de acerto e o tempo de resposta dos usuários durante a resolução dos exercícios. Embora essas abordagens permitam investigar aspectos distintos do aprendizado, as diferenças entre os públicos, os conteúdos e os métodos experimentais não permitem comparar diretamente seus resultados. Dessa forma, os trabalhos relacionados fornecem referências para a concepção e a avaliação de sistemas de ensino, enquanto o Shogi-sensei se diferencia por reunir exercícios direcionados aos fundamentos do shogi, recomendados com base no desempenho do usuário em cada habilidade específica.

Capítulo 4

Desenvolvimento

O sistema do Shogi-sensei pode ser dividido em quatro blocos principais: o banco de dados, a *engine* do YaneuraOu, o servidor e por fim a interface. A relação entre esses componentes é apresentada na Figura 4.1. Cada bloco foi desenvolvido em um *docker container* diferente, com o intuito de permitir que o projeto possa ser executado em qualquer máquina.

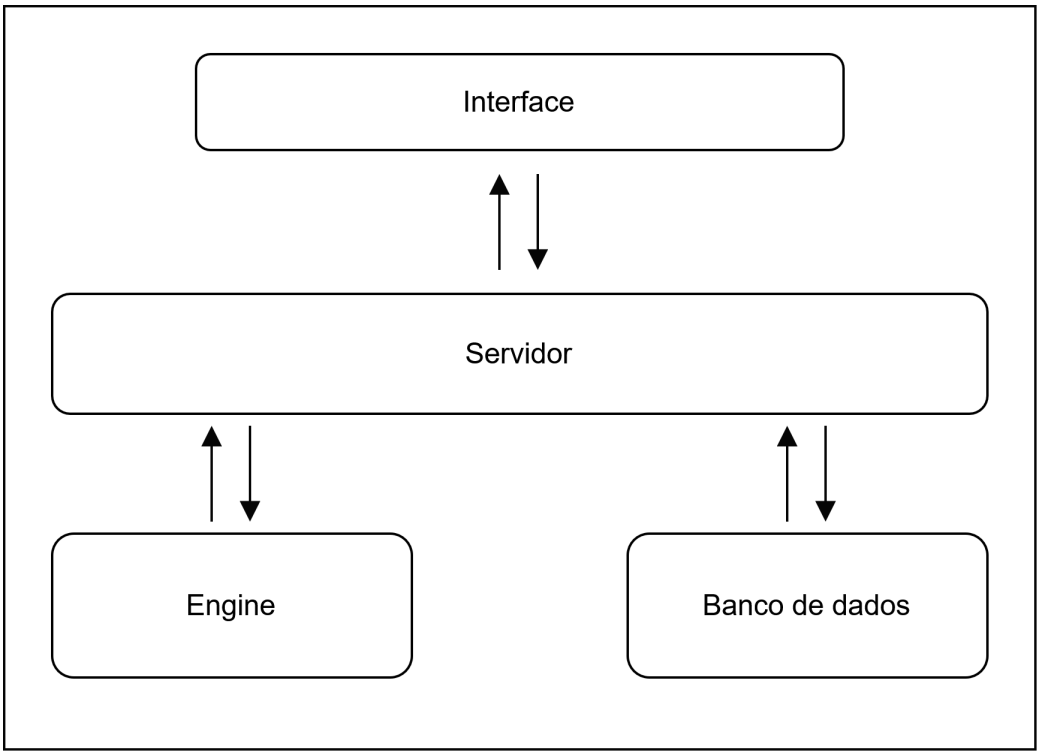


Figura 4.1: *Visão geral do sistema*

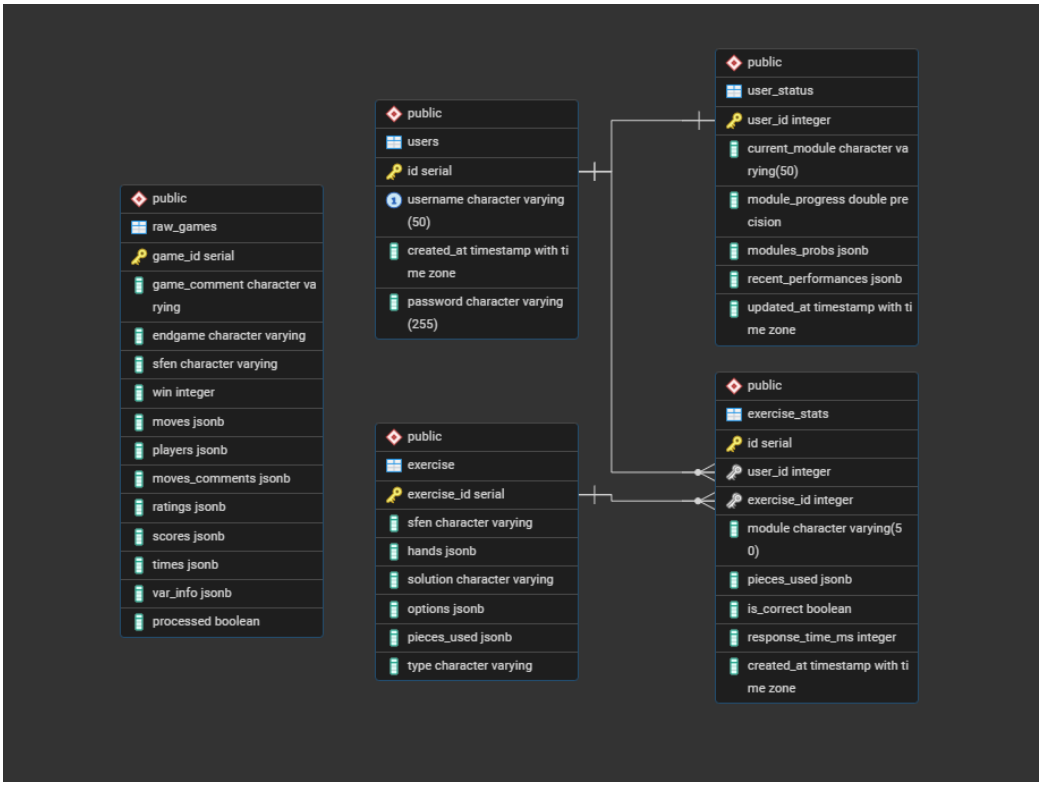


Figura 4.2: *Tabelas do banco de dados*

4.1 Banco de dados

O sistema utiliza o *PostgreSQL* para armazenar informações dos usuários e dos exercícios. A escolha desse sistema foi motivada pela experiência prévia do autor com a ferramenta e pela facilidade de integração com Python por meio de bibliotecas como *SQLAlchemy*, utilizada para definir modelos e realizar operações sobre os dados; *psycopg*, responsável pela comunicação com o banco de dados; e *Alembic*, empregado para gerenciar alterações no esquema do banco por meio de migrações. As tabelas são apresentadas na Figura 4.2 e são descritas abaixo:

- **raw_games**: armazena os dados brutos de partidas extraídas da plataforma WDoor[*Plataforma de partidas wdoor* s.d.], que disponibiliza partidas no formato CSA, apresentado na seção 4.3.
- **users**: armazena os dados de cadastro dos usuários do sistema, incluindo o hash da senha, em vez da senha original.
- **user_status**: contém informações referentes ao progresso do usuário.
  - **current_module**: identifica o módulo em que o usuário se encontra.
  - **module_progress**: registra o progresso do usuário no módulo atual.
  - **modules_probs**: armazena a distribuição de probabilidades utilizada para orientar a recomendação dos módulos de exercícios.

  - **recent_performances**: mantém um histórico do desempenho do usuário nos módulos.

- **exercise**: contém os dados dos exercícios gerados.
  - **sfen**: representação SFEN do estado do tabuleiro.
  - **hands**: peças que cada jogador tem em mãos.
  - **solution**: solução do exercício.
  - **options**: alternativas do exercício, que incluem a solução.
  - **pieces_used**: peças utilizadas na solução do exercício.
  - **type**: módulo ao qual o exercício pertence.
- **exercise_stats**: armazena as estatísticas das respostas enviadas pelos usuários para os exercícios.
  - **user_id**: identificação única do usuário.
  - **exercise_id**: identificação única do exercício.
  - **module**: módulo ao qual o exercício pertence.
  - **pieces_used**: mesmo campo presente na tabela **exercise**.
  - **is_correct**: indica se a resposta foi correta (True) ou incorreta (False).
  - **response_time**: tempo, em milissegundos, gasto pelo usuário para responder ao exercício.

4.2 *Engine*

Assim como mencionado no início desta seção, o sistema utiliza a *engine* YaneuraOu, baseada em um modelo NNUE (*Efficiently Updatable Neural Network*). Escolheu-se essa *engine* por não depender de uma placa de vídeo poderosa, o que viabilizou usá-la num ambiente local. Como a comunicação com o YaneuraOu ocorre através do envio de comandos USI pela entrada do próprio processo e lê-se a saída devolvida, optou-se por separá-lo do servidor. Assim, utilizou-se a biblioteca *FastAPI* para criar um serviço que realiza a comunicação entre os dois processos, com o benefício do servidor não ser prejudicado quando a *engine* apresentar algum erro.

```python
class Engine():
    def __init__(self, model: str):
        self.verbose = True
        self.model = model
        self.lock = threading.Lock()
        self.engine = self.start_yaneuraou_engine(model)

    def start_yaneuraou_engine(self, model: str):
        engine = subprocess.Popen(
            [model],
            stdin=subprocess.PIPE,
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            text=True,
            bufsize=1,
            cwd="/app/"
        )

        engine.stdin.write("usi\n")
        engine.stdin.flush()
        self.read_until(engine, "usiok")
        print("Engine ok")
        engine.stdin.write("setoption name EvalDir value /app/eval\n")
        engine.stdin.flush()
        engine.stdin.write("setoption name USI_Hash value 32\n")
        engine.stdin.flush()

        engine.stdin.write("setoption name Threads value 8\n")
        engine.stdin.flush()
        engine.stdin.write("setoption name USI_OwnBook value false\n")
        engine.stdin.flush()
        engine.stdin.write("isready\n")
        engine.stdin.flush()
        self.read_until(engine, "readyok")
        print("Engine ready!")

        return engine
```

Figura 4.3: *Inicialização da engine*

Para inicializar o YaneuraOu, envia-se o comando *usi*. Após ele enviar a mensagem *usiok*, envia-se o comando *setoption name EvalDir value <path>*, onde *path* se refere ao caminho para o diretório que contém os pesos da rede neural utilizada na avaliação das posições. Em seguida, são realizadas algumas configurações de desempenho. Por fim, envia-se o comando *isready* e, após a *engine* enviar *readyok*, a inicialização está finalizada.

Embora inicialmente estivesse previsto o uso do YaneuraOu na geração de exercícios de todos os módulos, durante o desenvolvimento verificou-se que ele seria necessário apenas no módulo de xeque-mate em um lance, no qual é empregado para auxiliar na geração de alternativas de resposta. O processo de geração dessas alternativas é apresen-

tado na seção 4.3.

4.3 Servidor

Para o desenvolvimento do servidor, utilizou-se a biblioteca *FastAPI*, responsável pela criação e pelo gerenciamento dos endpoints da API consumida pela interface do sistema. Esse componente ficou responsável pelas seguintes funcionalidades:

- Gerenciar a comunicação com o banco de dados.
- Realizar o cadastro e a autenticação dos usuários;
- Gerar os exercícios automaticamente;
- Organizar os exercícios em listas;
- Corrigir os exercícios e gerar a explicação;
- Atualizar e consultar as estatísticas de desempenho dos usuários;

Em ambiente de produção, o servidor foi hospedado em um celular antigo, com o objetivo de reduzir o risco de interrupções causadas por eventuais quedas de energia durante os experimentos, uma vez que o dispositivo pode ser conectado a uma bateria externa para garantir sua autonomia durante esses períodos. Para disponibilizar o servidor para conexões externas, utilizou-se o aplicativo *Termux*, que fornece um ambiente Linux no dispositivo. Em seguida, foram instalados o *PostgreSQL* e as dependências necessárias para executar o projeto. Por fim, utilizou-se o serviço *Tailscale Funnel* para disponibilizar o servidor por meio de uma URL pública e estável. Embora essa solução tenha permitido a comunicação entre a interface e o servidor, as limitações de hardware do dispositivo afetaram o desempenho da aplicação, resultando em tempos de resposta mais elevados.

4.3.1 Arquitetura

O servidor utiliza uma arquitetura em camadas, com o objetivo de separar as responsabilidades dos componentes do sistema, facilitando sua manutenção e isolando a implementação de cada funcionalidade. Dessa forma, o código foi organizado em três camadas:

- Repositórios: responsáveis por realizar as operações de persistência e consulta de dados no banco de dados.
- Serviços: responsáveis por implementar e gerenciar a lógica de funcionamento do sistema.
- Roteadores: responsáveis por definir os endpoints da API, recebendo as requisições dos clientes e encaminhando-as aos serviços correspondentes.

Durante o processamento de uma requisição, o roteador aciona o serviço responsável pela funcionalidade solicitada, que, quando necessário, utiliza os repositórios para acessar o banco de dados. O fluxo é representado na Figura 4.4.

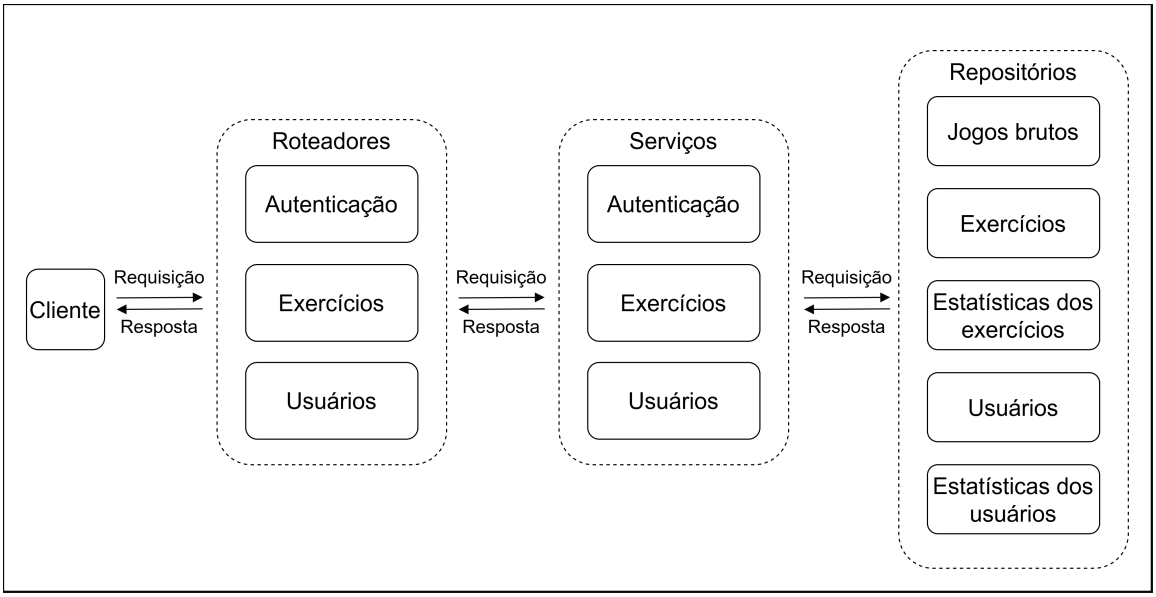


Figura 4.4: *Arquitetura do servidor*

Além dessas camadas, foram utilizados esquemas do Pydantic para estruturar e validar os dados recebidos e retornados pelos endpoints, garantindo um formato padronizado para a comunicação entre o cliente e o servidor.

4.3.2 Autenticação

O *FastAPI* também oferece recursos para implementar mecanismos de segurança em uma aplicação. Nesse sentido, o sistema desenvolvido utiliza o padrão OAuth2 para gerenciar a autenticação dos usuários, garantindo que cada um tenha acesso apenas aos próprios dados. Para isso, são utilizados tokens JWT (*JSON Web Token*), que permitem validar as requisições subsequentes sem a necessidade de realizar um novo login a cada acesso. Por fim, as senhas cadastradas são armazenadas na forma de hashes, gerados e verificados por meio da biblioteca *pwdlib*.

4.3.3 Geração automática de exercícios

Para padronizar a apresentação dos exercícios e simplificar a sua implementação na interface, todas as questões consideram que é a vez do jogador *sente* de jogar. Cada questão apresenta quatro alternativas, das quais apenas uma é a correta, e pertence a um dos seguintes módulos:

- **Reconhecimento**: pede para que o usuário identifique uma determinada peça no tabuleiro.
- **Movimento 1**: apresenta apenas uma peça sem o símbolo escrito no tabuleiro com seus movimentos destacados e pede para que o usuário identifique qual seria tal peça.
- **Movimento 2**: semelhante ao Movimento 1, mas existem outras peças no tabuleiro que impedem alguns movimentos.

- **Promoção**: pede para que o usuário identifique qual movimento pode levar a uma promoção.
- **Drop**: pede para que o usuário identifique qual movimento constitui um *drop* legal.
- **Xeque-mate**: pede para que o usuário identifique qual movimento leva ao xeque-mate.

Assim como mencionado na seção 4.1, os exercícios são gerados a partir de uma plataforma que contém partidas entre jogadores reais, com o intuito de garantir que as posições apresentadas no tabuleiro são legais e expor os usuários do sistema a cenários encontrados em partidas reais. Esses jogos são disponibilizados no formato CSA, arquivos que contêm informações como nome dos jogadores, registro dos movimentos, estado do tabuleiro ao final da partida no formato SFEN e qual foi o vencedor. Para processar os dados dos jogos, foi utilizada uma biblioteca chamada *cshogi*[*cshogi* s.d.], que permite manipular um tabuleiro de shogi, extrair informações, como a posição das peças e as mãos de cada jogador a partir da representação SFEN, simular movimentos e verificar a legalidade das jogadas. Dessa forma, para cada jogo, a geração segue os passos abaixo:

1. Simulação dos movimentos registrados;
2. Para cada movimento, são definidas a solução e as opções de acordo com os critérios de cada módulo.
3. Registro desse exercício em uma lista para o posterior registro no banco de dados.

Mais detalhes do passo 2 estão descritos abaixo:

- **Reconhecimento**: A solução consiste em selecionar aleatoriamente uma peça no tabuleiro. As alternativas são compostas pela solução e outras três peças diferentes. Por exemplo, se um peão é escolhido como solução, as outras alternativas podem ser lança, cavalo e rei.
- **Movimento 1**: Nesse módulo, não é necessário utilizar os jogos extraídos da plataforma. Para cada uma das 14 peças possíveis, é definido um tabuleiro que contém apenas essa peça em sua região central, representado pelo SFEN 9/9/9/9/<peça>/9/9/9/9. Em seguida, as alternativas são selecionadas entre as peças restantes que apresentam padrões de movimento diferentes do padrão da solução, evitando a possibilidade de mais de uma resposta correta. Por exemplo, caso a solução seja um general de ouro, as alternativas não poderão incluir peças promovidas, com exceção da torre e do bispo promovidos, pois as demais possuem o mesmo padrão de movimento do general de ouro.
- **Movimento 2**: Diferentemente do módulo Movimento 1, uma peça presente no tabuleiro de uma partida real é selecionada aleatoriamente para ser a solução. As alternativas são determinadas seguindo o mesmo critério do módulo anterior. Entretanto, a presença de outras peças no tabuleiro pode restringir os movimentos disponíveis, adicionando uma camada de dificuldade, pois nem todos os movimentos possíveis de uma peça estarão necessariamente disponíveis na posição apresentada.
- **Promoção**: Inicialmente, é selecionado aleatoriamente um movimento que permite a promoção de uma peça. Durante a seleção das alternativas, são descartados os

movimentos que também resultariam na promoção de uma peça. Além disso, para cada movimento de promoção descartado, também é eliminado o movimento correspondente sem promoção, mesmo que nenhum dos dois corresponda à solução. Isso evita que duas alternativas representem a mesma jogada, diferenciando-se apenas pela decisão de promover ou não a peça. Por exemplo, se a solução for 8d->8c+ e uma das alternativas possíveis for 7d->7c+, esta última será descartada, assim como 7d->7c.

- **Drop**: O processo é semelhante ao módulo Promoção. Um movimento de drop é escolhido aleatoriamente e definido como a solução do exercício. Em seguida, os demais movimentos de drop legais são descartados. Por fim, as alternativas são constituídas por movimentos de drop ilegais, como o posicionamento de um peão em uma coluna que já contém outro peão aliado.

- **Xeque-mate**: Verifica-se se determinado movimento resulta em xeque-mate. Caso isso ocorra, o movimento identificado e o estado do tabuleiro são enviados para a *engine*, responsável pela geração das alternativas. Seu uso nesse processo permite selecionar movimentos candidatos com base na posição, tornando as alternativas mais plausíveis e, consequentemente, aumentando a dificuldade do exercício. Isso evita que movimentos claramente distantes da região onde se encontra o rei adversário evidenciem facilmente a resposta correta. Por exemplo, suponha que a solução seja mover um general de ouro da casa 4d para a casa 5c. Caso as alternativas incluam movimentos de peões para casas distantes do rei adversário, como as linhas g ou h, torna-se mais fácil identificar a solução. No entanto, é possível que dentre os movimentos escolhidos pelo YaneuraOu, exista mais de um xeque-mate. Dessa forma, esses movimentos são também excluídos.

4.3.4 Listas de exercício

As listas geradas pelo sistema contêm 10 questões. Inicialmente, apenas exercícios do módulo Reconhecimento estão disponíveis. A seleção das questões segue uma distribuição de probabilidades entre os módulos desbloqueados, que é atualizada de acordo com o desempenho do usuário.

Quando uma lista é submetida para correção, as questões respondidas incorretamente são enviadas em uma única requisição a um LLM, responsável por gerar explicações sobre os motivos pelos quais as respostas fornecidas estão incorretas. Foram testados três modelos diferentes, utilizando o mesmo prompt (Figura 4.5) e os planos gratuitos. Considerou-se satisfatória uma resposta quando ela explicava corretamente os conceitos necessários para compreender o erro cometido. Embora o *Gemini 3.6-flash* tenha gerado respostas satisfatórias, seu limite de apenas 20 requisições diárias restringiria a correção a 20 listas por dia. Considerando a quantidade de usuários prevista para o experimento, esse limite foi considerado insuficiente. Em seguida, testou-se o *Gemma 4 26B*, que permite até 14.400 requisições diárias, mas apresentou um tempo de resposta elevado. Por fim, optou-se pelo *Gemini 3.5-flash-lite*, que apresentou respostas satisfatórias e oferece um limite de 500 requisições diárias.

```
A sua tarefa é corrigir exercícios de Shogi. Você receberá o tabuleiro seguindo o formato SFEN,
assim como a resposta do usuário e a resposta esperada. Você deve explicar porque o usuário
errou a resposta, mostrando de forma clara os motivos pelos quais a resposta esperada é a
correta e considerando que o usuário é um iniciante.

As peças se movem da seguinte forma:
  - Peão: anda uma casa para frente
  - Cavalo: anda em L apenas para frente
  - Lança: anda quantas casas quiser, mas apenas para frente
  - General de Prata: anda uma casa para frente ou uma casa para cada diagonal
  - General de Ouro: anda uma casa para todos os lados (cima, baixo, esquerda e direita)
    ou cada diagonal da frente
  - Bispo: anda quantas casas quiser para todas as diagonais
  - Torre: anda quantas casas quiser em para todos os lados (cima, baixo, esquerda e direita)
  - Rei: anda uma casa para todos os lados e diagonais
  - Bispo promovido: anda como um Bispo, mas também passa a poder andar uma casa para todos os lados
    (cima, baixo, esquerda e direita)
  - Torre promovida: anda como uma torre, mas também passa a poder andar uma casa para todas as diagonais
  - As outras peças promovidas passam a se movimentar como um General de Ouro

Um movimento de drop só pode ser feito quando:
  - A peça colocada não consegue fazer um movimento em turnos futuros (peões e lanças na ultima
    linha; cavalos nas últimas 2 linhas);
  - A casa já está ocupada;
  - A peça é um peão e o posicionamento leva a um chequemate;
  - A peça é um peão e na coluna já há um peão aliado.

A promoção pode ocorrer quando a origem ou destino do movimento está dentro da zona de promoção,
que corresponde às três fileiras mais distantes do jogador. Em determinadas situações ela é
obrigatória: por exemplo, um peão ou uma lança que alcança a última fileira, ou um cavalo que
alcança uma das duas últimas fileiras, pois não possuem movimentos legais a partir dessas posições.

Existem os seguintes módulos:
  - recon: Envolve o reconhecimento das peças. A pergunta é "Qual é a peça na posição X?". A posição X
           vem acompanhada pela resposta esperada. Quando explicar essa questão, ao se referir a resposta
           esperada, não use o formato peça:posição. Por exemplo: caso o solução seja Rei:2c, a explicação
           deve ser algo como "Você escolheu peça X, mas a peça na posição 2c é o Rei.". Se certifique de
           mencionar as posições das peças na solução e na resposta dada pelo usuário. Aém disso, foque na
           diferença visual entre os kanjis nas peças.
  - movement1 e movement2: Envolvem o reconhecimento dos movimentos das peças. A pergunta é "Qual peça
                           possui os movimentos destacados?". Os movimentos vem acompanhados pela resposta
                           esperada.
  - checkmate-in-one: Envolve identificar qual jogada levará ao chequemate. A pergunta é "Qual movimento
                      leva ao chequemate?". Explique detalhadamente porque a escolha do usuário está errada
                      e porque a solução é a resposta esperada.
  - promotion: Envolve identificar qual movimento possibilita a peça a ser promovida. A pergunta é "Qual
               movimento pode resultar numa promoção?". Mencione a peça que pode ser promovida e qual peça ela
               pode virar. Explique as regras da promoção.
  - drop: Envolve identificar qual movimento de drop é válido. A pergunta é "Qual é a casa do tabuleiro
          na qual você pode corretamente posicionar a peça X da sua mão?". Ao explicar a questão, foque apenas
          em deixar claro o motivo pelo qual a jogada é ilegal e ignore a estratégia por trás da jogada.
          Lembre de mencionar a resposta esperada.
```

Figura 4.5: *Prompt para o LLM gerar a explicação*

Após a geração das explicações, o desempenho do usuário é calculado e então a distribuição de probabilidades dos módulos é atualizada de acordo com os seguinte processo:

1. Calcula-se a taxa de acerto do usuário em cada módulo;

2. Caso a taxa de acerto seja inferior a 70%, a probabilidade de seleção do módulo é multiplicada por 1,2. Caso contrário, é multiplicada por 0,75;

3. Caso o usuário tenha realizado pelo menos 10 listas do módulo atual e obtido uma taxa de acerto média de, no mínimo, 70%, o módulo seguinte é desbloqueado e recebe uma probabilidade de 80% de ser selecionado;

4. Por fim, as probabilidades são normalizadas, garantindo que a soma da distribuição seja igual a 100%.

4.3.5 Interface

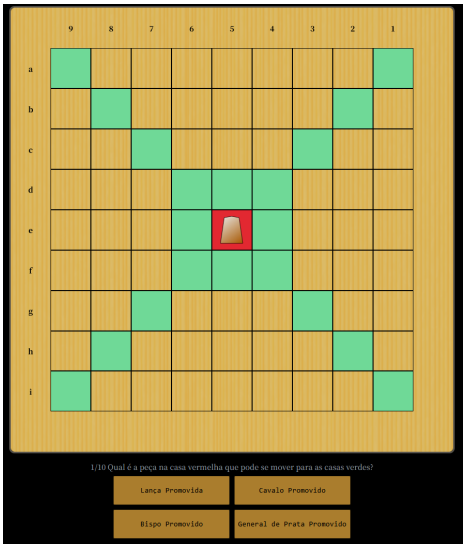
A estrutura da interface foi desenvolvida utilizando o *React*, que permite organizar o sistema em componentes reutilizáveis e facilita a implementação de um layout responsivo. Além disso, o *Tailwind CSS* foi utilizado para simplificar a estilização dos componentes.

Foram desenvolvidas cinco telas principais: inicial, cadastro, login, exercícios e resultados. A tela inicial permite acessar as funcionalidades disponíveis, enquanto as telas de cadastro e login são responsáveis pela autenticação dos usuários. A interface se adapta às dimensões da tela do dispositivo, permitindo a utilização do sistema tanto em computadores quanto em dispositivos móveis.

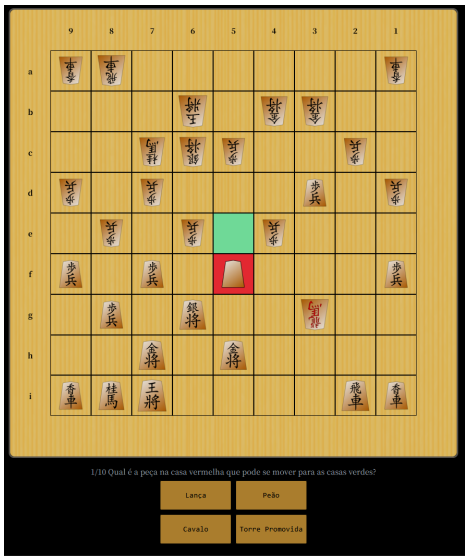
Na tela de exercícios, o usuário pode resolver as questões de cada lista selecionando a alternativa que julgar correta. Após o envio da lista, a tela de resultados apresenta o desempenho obtido e as explicações geradas para as questões respondidas incorretamente.



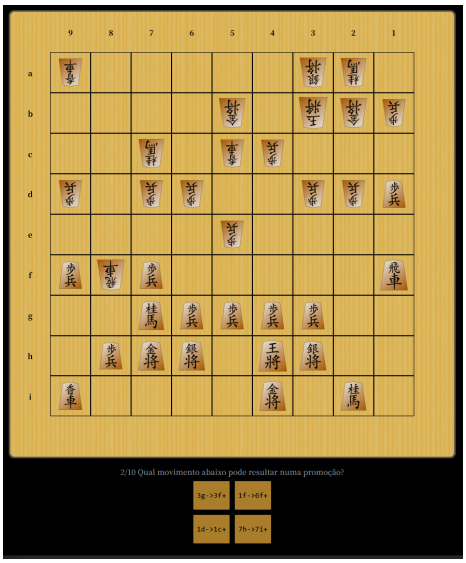
(a) *Reconhecimento*



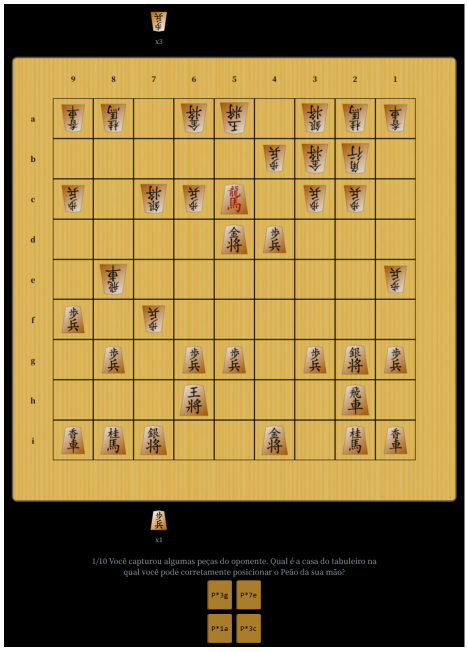
(b) *Movimento 1*



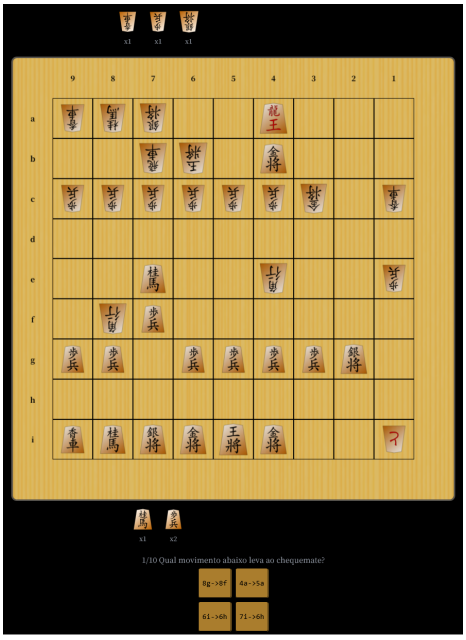
(c) *Movimento 2*



(d) *Promoção*



(e) *Drop*



(f) *Xeque-mate*

Figura 4.6: *Exemplos das interfaces dos exercícios*

Capítulo 5

Metodologia

Para avaliar o sistema proposto neste trabalho, foi realizado um experimento com 15 participantes sem conhecimento prévio de shogi, durante um período de três semanas. Ao longo desse período, os participantes foram instruídos a resolver pelo menos uma lista de exercícios por dia, utilizando o sistema.

Durante o experimento, foram coletadas métricas relacionadas ao desempenho dos participantes, incluindo o tempo de resposta, a taxa média de acertos por módulo e a quantidade de listas necessárias para avançar para o módulo seguinte. Essas métricas permitiram analisar o desempenho dos usuários e o funcionamento da progressão de dificuldade implementada no sistema.

Ao final do experimento, foi disponibilizado um questionário para coletar as opiniões dos participantes sobre diferentes aspectos do sistema, como usabilidade, organização dos módulos, dificuldade dos exercícios e contribuição para o aprendizado. O questionário foi composto pelas seguintes perguntas:

- De 1 a 10, você considera a interface intuitiva e fácil de utilizar?
- Qual foi o módulo que você teve mais dificuldade?
- Qual o motivo?
- A progressão dos módulos faz sentido? Por que?
- Você considera que a quantidade de exercícios por lista é a ideal? Por que?
- Você considera que aprendeu os fundamentos básicos do shogi após utilizar o sistema? Por que?
- De 1 a 10, você considera que as explicações apresentadas pelo sistema são claras e fáceis de compreender?
- De 1 a 10, o quanto você considera que as explicações ao final das listas contribuiu para o seu aprendizado?
- Você tem alguma sugestão de pontos a melhorar no sistema?

Capítulo 6

Resultados

Capítulo 7

Conclusão

Referências

[*cshogi* s.d.] *cshogi*. URL: https://github.com/TadaoYamaoka/cshogi (citado na pg. 19).

[Ito 2018] Takeshi Ito. "Game learning support system based on future position". *ICGA Journal* 40.4 (2018), pp. 450–459. DOI: 10.3233/ICG-180068. eprint: https://journals.sagepub.com/doi/pdf/10.3233/ICG-180068. URL: https://journals.sagepub.com/doi/abs/10.3233/ICG-180068 (citado na pg. 9).

[Kochmar *et al.* 2020] Ekaterina Kochmar *et al.* "Automated personalized feedback improves learning gains in an intelligent tutoring system". In: *Artificial Intelligence in Education*. Ed. por Ig Ibert Bittencourt, Mutlu Cukurova, Kasia Muldner, Rose Luckin e Eva Millán. Cham: Springer International Publishing, 2020, pp. 140–146 (citado na pg. 11).

[Hodges s.d.] George Hodges. *Shogi Magazine, vol 2, p.3,* (citado na pg. 3).

[*Plataforma de partidas wdoor* s.d.] *Plataforma de partidas wdoor*. URL: https://wdoor.c.u-tokyo.ac.jp/shogi/ (citado na pg. 14).

[*Plataforma Korbit* s.d.] *Plataforma Korbit*. URL: https://www.korbit.ai (citado na pg. 11).

[*Repositório do YaneuraOu* s.d.] *Repositório do YaneuraOu*. URL: https://github.com/yaneurao/yaneuraou (citado na pg. 7).

[Romstadt s.d.] Tord Romstadt. "The universal shogi interface" (citado na pg. 5).

[Gilliard s.d.] Garret Gilliard. *An Introduction to Shogi* (citado na pg. 4).

[M. Zhang *et al.* 2024] Mingyu Zhang, Qiao Jin, Qian Dong, Danli Wang e Jun Xie. "Ek-chess: chess learning system based on top-level chess expert knowledge graph". *International Journal of Human-Computer Interaction* (mai. de 2024), pp. 1–15. DOI: 10.1080/10447318.2024.2348229 (citado na pg. 10).

[Q. Zhang *et al.* 2025] Qiao Zhang, Chunyi Wang e Christopher J. MacLellan. "(a)i can play gomoku: an intelligent tutoring system for strategic games". In: *Companion Proceedings of the Annual Symposium on Computer-Human Interaction in Play*. CHI PLAY Companion '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 239–244. DOI: 10.1145/3744736.3749191 (citado na pg. 9).